Software Engineering Lab

Lab 15

Load Testing

Baca Antrean,
Pahami Tail Latency



lukman-ss

Booking cepat saat sepi, antre saat ramai

Satu request sukses belum menggambarkan pengalaman banyak pengguna yang datang bersamaan.

JALUR LAB

Goroutine VU → HTTP POST /booking → semaphore → timer → respons 201.

PERTANYAAN

Bagaimana latency berubah ketika peminta lebih banyak daripada slot kerja?

Satu booking yang berhasil belum menjelaskan perilaku endpoint ketika banyak pengguna datang bersama. Lab 15 mengirim HTTP POST ke /booking pada server lokal. Perbandingan beban rendah dan tinggi membantu melihat perubahan latency ketika slot kerja terbatas. Fokusnya adalah antrean dan distribusi waktu respons, bukan integrasi database production atau pembuktian bahwa aplikasi siap melayani semua pola traffic.

A successful request is not a capacity test.

Dua skenario dengan server yang sama

Lima slot, query dasar 20 ms. Smoke dan stress masing-masing berjalan dua detik.

Parameter	Smoke	Stress
Virtual users	2	50
Slot server	5	5
Durasi	2 detik	2 detik

Demo memakai lima slot simulasi database dan query dasar 20 ms. Smoke menjalankan dua virtual user, sedangkan stress menjalankan 50 virtual user. Keduanya berdurasi dua detik, dijalankan berurutan, memakai endpoint dan body booking yang sama. Angka tersebut merupakan konfigurasi eksperimen kecil untuk memperlihatkan saturasi; jumlah VU tidak otomatis menjadi target pengguna bisnis atau kapasitas maksimum layanan.

50 VU adalah concurrency konfigurasi, bukan 50 RPS.

VU mengirim request secara berurutan

Satu VU menunggu hasil request sebelum memulai request berikutnya.

CLOSED LOOP

Respons makin lambat → putaran VU makin lama → request per detik ikut berubah.

BATAS MODEL

Tidak ada fixed arrival rate, think time, ramp-up, atau skenario multi-endpoint.

Runner membuat satu goroutine untuk setiap VU. Di dalamnya, request berikutnya dikirim setelah request sebelumnya selesai atau gagal. Tidak ada think time, ramp-up, atau penjadwal arrival rate. Ini menghasilkan pola closed loop: ketika respons melambat, setiap VU mengirim lebih sedikit request dalam satu detik. Karena itu, menaikkan VU dan menetapkan RPS adalah dua eksperimen yang berbeda.

Concurrency dan arrival rate perlu dibaca sebagai parameter berbeda.

Batas kerja memakai buffered channel

Handler yang belum memperoleh slot menunggu. Pemegang slot dibatasi kapasitas channel.

```
select {
case s.semaphore <- struct{}{}:
case <-r.Context().Done():
return
}
defer func() { <-s.semaphore }()
```

Server membuat semaphore berupa buffered channel dengan kapasitas MaxDBConnections. Handler harus memasukkan token sebelum melanjutkan simulasi query. Ketika channel penuh, handler menunggu token lain dilepas. Defer mengembalikan slot saat handler keluar. Mekanisme ini membatasi pekerjaan yang sedang memegang slot, tetapi tidak menyediakan antrean eksplisit dengan batas panjang atau kebijakan penolakan ketika antrean terlalu besar.

Yang dibatasi adalah slot aktif, bukan seluruh request yang menunggu.

Slot database adalah simulasi

Nama config menggambarkan resource simulasi. Timer menggantikan query database.

TERSEDIA

ActiveConnections() membaca jumlah slot semaphore yang sedang digunakan.

BELUM DIUKUR

Panjang antrean, pool wait histogram, query plan, CPU, memory, dan I/O.

Nama MaxDBConnections menggambarkan resource yang disimulasikan. Tidak ada driver SQL, query nyata, tabel booking, atau transaksi database pada implementasi ini. Timer mewakili durasi kerja. ActiveConnections mengembalikan len(semaphore), yaitu jumlah slot yang telah diakuisisi. Nilai itu bukan ukuran panjang antrean, bukan jumlah koneksi TCP, dan bukan telemetry dari database sebenarnya. Interpretasi metrik harus mengikuti objek yang diukur.

Slot penuh dapat diamati; lokasi bottleneck production perlu bukti tambahan.

Antrean ditambah tail spike acak

Saat activeReq > kapasitas, peluang 10% membuat query simulasi berlangsung 25 kali lebih lama.

```
if atomic.LoadInt64(&s.activeReq) >
int64(s.cfg.MaxDBConnections) {
if rand.Float32() < 0.10 {
dur = s.cfg.DBQueryDuration * 25
}
}
```

Handler menambah activeReq sebelum memperoleh slot, sehingga counter mencakup handler POST yang sedang menunggu maupun bekerja. Setelah slot diperoleh, jika activeReq melebihi kapasitas, ada peluang sepuluh persen durasi query menjadi 25 kali durasi dasar. Pada konfigurasi demo, itu berarti 500 ms. Spike ini sengaja dimasukkan ke model; hasil stress tidak berasal dari antrean murni dengan service time konstan.

20 ms × 25 = 500 ms: nilai konfigurasi, bukan benchmark baru.

Context membatasi waktu hidup tes

Duration menjadi timeout context bersama; client memiliki timeout lima detik.

RUNNER

Request memakai context tes. Parent context dapat mengakhirinya lebih awal.

SERVER

Select memeriksa cancellation saat antre slot dan saat timer query berjalan.

Run membungkus context caller dengan context.WithTimeout menggunakan Duration. Context tersebut dipasang pada setiap request HTTP. Client juga memiliki timeout lima detik, sehingga batas caller atau durasi tes dapat mengakhiri request lebih dulu. Handler server memeriksa cancellation saat menunggu slot dan saat menunggu timer. Ini memberi jalur keluar dari penantian, tetapi tidak mengubah cancellation menjadi respons overload dengan status khusus.

Durasi tes dan timeout client dapat memotong request yang masih berjalan.

Cancellation tidak seluruhnya masuk total

Error dari NewRequest atau client.Do dihitung hanya ketika context tes belum selesai.

```
if err != nil {
if ctx.Err() == nil {
errs++
}
continue
}
```

Ketika pembuatan request atau client.Do gagal, runner hanya menambah error jika ctx.Err masih nil. Jadi request yang dibatalkan karena durasi tes habis dapat dikeluarkan dari hitungan. TotalRequests adalah jumlah latency sukses ditambah error yang tercatat, bukan penghitung seluruh request yang pernah dimulai. Perbedaan ini penting ketika membandingkan jumlah attempt, throughput selesai, dan jumlah request yang tertinggal di akhir eksperimen.

TotalRequests bukan seluruh started attempts.

Definisi sukses mengikuti status HTTP

Kode memakai status >=400 sebagai error. Status <400 menghasilkan sampel latency.

Hasil	Pencatatan
HTTP >=400	ErrorCount bertambah
HTTP <400	Latency + SuccessCount
Body JSON bisnis	Tidak divalidasi

Setelah memperoleh respons, runner membaca body sampai selesai lalu menutupnya. Status minimal 400 menambah error; status di bawah 400 masuk latency sukses. Artinya aturan kode lebih luas daripada hanya 2xx, walaupun handler booking normal menghasilkan 201. Runner tidak memvalidasi JSON status confirmed atau booking_id. Body read error juga diabaikan. Label SuccessCount perlu dipahami sebagai hasil aturan teknis runner, bukan validasi transaksi booking.

HTTP success menurut runner belum membuktikan booking bisnis valid.

Latency berakhir setelah body dibaca

Timer runner mencakup client.Do dan konsumsi response body untuk request sukses.

UKURAN

Latency HTTP yang diamati client; bukan waktu query database terpisah.

BATAS DIAGNOSIS

Tidak ada breakdown DNS, connect, TLS, TTFB, atau waktu antre slot.

reqStart dicatat sebelum client.Do. Untuk respons yang dianggap sukses, time.Since dicatat sesudah body didrain dan ditutup. Angka ini mencakup jalur HTTP client sampai konsumsi body, bukan durasi timer query saja. Runner tidak memisahkan DNS, dial, TLS, time to first byte, pool wait, atau transfer body. Karena itu, kenaikan latency agregat belum langsung menentukan lapisan mana yang memakan waktu.

Latency agregat menunjukkan gejala; belum mengisolasi semua penyebab.

Percentile memakai subset sukses

Latency request gagal tidak disimpan. Semua percentile memakai subset respons sukses.

Kondisi	Interpretasi
SuccessCount > 0	Percentile subset sukses
Semua request error	Latency fields tetap nol
P95 = 0 tanpa sukses	Bukan bukti layanan cepat

Respons error dan transport error menambah counter tanpa menyimpan latency. CalculateMetrics menerima slice latency sukses dan jumlah error terpisah. P50, P90, P95, P99, min, max, dan average dihitung dari slice tersebut saja. Jika semua request gagal, statistik latency tetap nol walaupun mungkin ada waktu tunggu panjang. Nol pada P95 saat SuccessCount nol tidak boleh dibaca sebagai layanan yang sangat cepat.

Baca percentile bersama SuccessCount dan ErrorCount.

RPS mencakup error yang tercatat

RPS = (sampel sukses + error tercatat) / durasi aktual dalam detik.

```
total := len(latencies) + errors
res.RPS = float64(total) /
totalDuration.Seconds()
```

CalculateMetrics menetapkan total sebagai len(latencies) ditambah errors. RPS membagi total itu dengan durasi aktual Run, yang dicatat setelah semua goroutine selesai. Maka RPS lab bukan success-only throughput. Bila tidak ada total request, fungsi mengembalikan Result kosong, termasuk Duration nol. Formula tersebut menjelaskan apa yang dilaporkan, tetapi tidak menyediakan target arrival rate maupun statistik request yang dibatalkan pada akhir tes.

RPS yang tinggi perlu dibaca bersama jumlah keberhasilan.

Percentile memilih indeks tanpa interpolasi

Rumus memakai indeks floor dan memilih satu sampel terurut.

```
idx := int(float64(len(sorted)-1) *
(pct / 100.0))
return sorted[idx]
```

Calculator menyalin slice lalu mengurutkannya, sehingga input tidak diubah. Percentile memilih indeks int((n-1) × p/100), dengan konversi integer membuang pecahan positif. Tidak ada interpolasi antara dua sampel. Untuk dataset test 1 sampai 100 ms, P50 menjadi 50 ms, P95 95 ms, dan P99 99 ms. Hasil ini sesuai algoritme lab, bukan klaim semua tool memakai definisi percentile yang sama.

Bandingkan metode percentile sebelum membandingkan angka antar-tool.

Fixture menjelaskan angka secara pasti

Contoh deterministik dari dataset unit test: 100 sampel, masing-masing 1–100 ms.

Metric	Hasil fixture
Min / max	1 ms / 100 ms
Average	50,5 ms
P50 / P95 / P99	50 / 95 / 99 ms

Seratus sampel 1 sampai 100 ms pada unit test menghasilkan minimum 1 ms, maksimum 100 ms, dan average 50,5 ms. time.Duration menyimpan unit nanosecond, sehingga pembagian tidak otomatis membulatkan ke millisecond utuh. Nilai fixture ini dapat dihitung langsung dari kode dan data test. Berbeda dengan output demo berbasis HTTP, ia bukan pengukuran performa mesin atau kapasitas endpoint booking.

Angka fixture menguji calculator, bukan kapasitas server.

Sampel kecil membuat tail kasar

Satu sampel 42 ms → P50, P95, dan P99 semuanya 42 ms.

RESOLUSI

Sedikit sampel membuat percentile berubah dalam lompatan besar.

KONTEKS

Cantumkan jumlah sampel sukses, durasi, dan rumus calculator.

Pada satu sampel 42 ms, seluruh percentile bernilai 42 ms. Pada jumlah sampel kecil, P95 atau P99 dapat menunjuk sampel yang sama dan bahkan tidak menunjuk maksimum karena rumus memakai n-1 lalu floor. Jadi label percentile tinggi tidak otomatis membuat estimasi tail stabil. Catat jumlah sukses, durasi, dan metode perhitungan agar pembaca tidak memberikan ketepatan lebih besar daripada data yang tersedia.

Label P99 tidak otomatis berarti tail terukur dengan presisi tinggi.

Average dan tail menjawab hal berbeda

Average merangkum; percentile memperlihatkan posisi dalam distribusi sampel sukses.

UNIT TEST

Min ≤ P50 ≤ P90 ≤ P95 ≤ P99 ≤ Max pada fixture test.

INTEGRATION TEST

Stress P95 > smoke P95, dan stress P95 > stress average.

Average merangkum seluruh latency sukses dalam satu nilai. P95 memilih bagian atas distribusi menurut rumus calculator. Unit test invariant memeriksa urutan min, P50, P90, P95, P99, dan max pada data yang mengandung spike. Integration test secara terpisah membandingkan P95 stress dengan average stress. Kedua bentuk assertion ini berbeda: urutan percentile tidak mengharuskan P95 selalu lebih besar daripada average untuk semua distribusi.

Assertion satu skenario jangan diubah menjadi hukum semua distribusi.

Angka demo adalah catatan run

Contoh output repository: smoke P95 ≈21,37 ms; stress P95 ≈1,357 detik.

Catatan	Arti
Sumber angka	engineering/03-execution-result.md
Status	Contoh run repository
ErrorCount = 0	Nol error yang dicatat runner

engineering/03-execution-result.md menyimpan contoh smoke P95 sekitar 21,37 ms dan stress P95 sekitar 1,357 detik, dengan error tercatat nol. Itu adalah hasil run yang dicatat repository, bukan pengukuran baru di sini. Random spike, scheduling, dan batas context membuat run dapat berubah. Error nol juga tidak berarti semua attempt selesai: cancellation menjelang akhir tes dapat tidak masuk ErrorCount maupun TotalRequests.

Hasil contoh tidak menjadi SLA atau jaminan rasio perlambatan.

Transport besar belum menjamin generator bebas bottleneck

Idle connection limits 1000 membantu reuse, tetapi bukan audit kapasitas mesin generator.

TRANSPORT

MaxIdleConns dan MaxIdleConnsPerHost mengatur penyimpanan koneksi idle.

METRIK

Semua sampel disimpan, digabung, disalin, lalu diurutkan; memori mengikuti jumlah sampel.

Runner memakai MaxIdleConns dan MaxIdleConnsPerHost bernilai 1000. Kedua field mengatur koneksi idle yang dapat disimpan; keduanya bukan bukti bahwa seluruh batas client atau mesin generator hilang. Lab tidak mengukur CPU generator, resource jaringan, atau overhead alokasi. Setiap VU menyimpan semua latency sukses dalam slice, kemudian data digabung dan disalin untuk sorting. Kebutuhan memori bertambah bersama jumlah sampel yang terkumpul.

Kapasitas generator tetap perlu diperiksa pada eksperimen lebih besar.

Booking response belum menjadi transaksi nyata

Respons normal: HTTP 201, status confirmed, booking_id tetap BK-1001.

IMPLEMENTASI

Body request tidak didecode; booking tidak disimpan ke database.

CAKUPAN

Simulasi cocok untuk belajar antrean HTTP, bukan validasi transaksi booking production.

Server menolak metode selain POST dengan 405. Untuk POST yang selesai, ia mengirim 201 dengan status confirmed dan booking_id tetap BK-1001. Body request tidak didecode menjadi BookingRequest dan tidak divalidasi. Tidak ada penyimpanan booking, autentikasi, idempotency, atau unique booking ID. Ini cukup untuk demonstrasi HTTP dan antrean, tetapi workload transaksi nyata perlu karakter data dan jalur kerja yang lebih representatif.

Payload terlihat seperti bisnis; jalur kerja tetap perlu dibaca dari kode.

Test memeriksa perilaku yang spesifik

Ada 5 unit test metrics dan 7 integration test pada snapshot yang dibaca.

METRIK

Fixture, input kosong, satu sampel, RPS, dan urutan percentile.

HTTP + RESOURCE

Stress, 500, 405, dial error, cancellation, batas slot, dan total = sukses + error.

Lima unit test metrics memeriksa fixture, input kosong, satu sampel, RPS, dan urutan statistik. Tujuh integration test memeriksa smoke versus stress, HTTP 500, metode salah, dial error, context cancellation, batas slot, dan invariant total. Test batas slot mem-poll ActiveConnections selama request concurrent serta memeriksa peak positif dan tidak melebihi kapasitas. Test tidak membuktikan fairness antrean atau menentukan kapasitas maksimum sistem.

Cakupan test mengikuti assertion, bukan semua klaim performance.

Bottleneck lab diketahui dari desain

Penyebab lab sengaja dimodelkan. P95 production saja belum menunjuk lapisan penyebab.

OBSERVASI LANJUTAN

Korelasikan latency dengan slot, pool wait, resource, dan dependency timing.

KRITERIA

Runtime belum memasang threshold SLA; tentukan target berdasarkan kebutuhan layanan.

Di lab ini, bottleneck slot sengaja dibangun di server. Dalam layanan nyata, kenaikan P95 saja belum memisahkan antrean aplikasi, database, jaringan, atau dependency eksternal. Materi lanjutan repository membahas korelasi metrik komponen dan target performa, tetapi runtime tidak memasang exporter, tracing, atau threshold SLA otomatis. Demo hanya mencetak ringkasan hasil. Diagnosis dan keputusan pass atau fail membutuhkan observasi serta kriteria tambahan.

Percentile menunjukkan gejala; component metrics membantu diagnosis.

Jalankan ulang, lalu baca hasil dalam konteks

Source dan assertion sudah diperiksa. Go tidak tersedia; test belum dijalankan ulang di sini.

```
cd labs/15-load-testing
go test -v ./...
go test -race ./...
go run ./cmd/demo
```

Modul menyatakan Go 1.22 dan menggunakan standard library. Perintah test, race detector, serta demo tersedia untuk dijalankan dari folder lab. Go belum tersedia di lingkungan penyusunan paket ini; source dan assertion dibaca tanpa menjalankan ulang test. Catatan passing test repository merupakan bukti run terdahulu. Mental model yang dibawa pulang: Load testing reveals how latency changes when demand competes for limited resources.

Load testing reveals how latency changes when demand competes for limited resources.

Eksperimen kecil, interpretasi yang teliti.

Load testing reveals how latency changes when demand competes for limited resources.

Software Engineering Lab 15 - Load Testing
internal/loadtest • internal/server
tests/loadtest_test.go • cmd/demo/main.go
README.md • go.mod

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/15-load-testing`

Lukman / lukman-ss

Sumber: commit 82af988b89439fa92b1200406c5afbd59ce72a56
Test dibaca, tidak dijalankan.
Slot database simulasi; contoh run bukan jaminan kapasitas production.